

Usando *filter*

`filter(função, iterável)` passa cada item pela função e mantém só os que dão **verdadeiro**. Como o `map`, devolve um iterador preguiçoso. Nesta aula: filtrar com lambda e com funções nomeadas, o atalho `filter(None, ...)`, o complemento `filterfalse`, filtros em listas de dicionários, a dupla `map` + `filter` e o equivalente em comprehension.

`filter(função, iterável)`

Função que diz sim/não

`filter(None, ...)`

`filterfalse`

Filtros encadeados

`map` + `filter`

Formação Claude Code 2026 · Baseado em docs.python.org (Built-in Functions: `filter`, `all`; `itertools.filterfalse`; Truth Value Testing; Functional Programming HOWTO), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Só passa *o que der True*

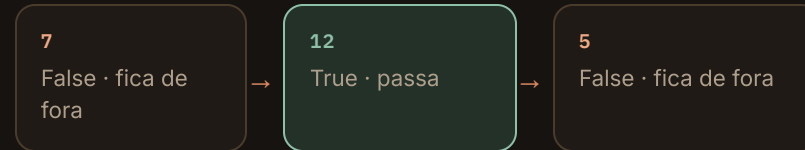
basico.py

```
numeros = [7, 12, 5, 30, 18, 3]

pares = filter(lambda n: n % 2 == 0, numeros)
print(pares)
print(list(pares))                # só os True passam
print(list(filter(lambda n: n > 15, numeros)))
```

SAÍDA

```
<filter object at 0x78f2e3dc05b0>
[12, 30, 18]
[30, 18]
```



- ✓ A função recebe **um item** e responde sim ou não
- ✓ O item passa inteiro: o `filter` não transforma nada
- ✓ O resultado é um iterador `filter`; `list()` mostra os itens

Regras com *nome*

funcao.py

```
def eh_primo(n):
    if n < 2:
        return False
    return all(n % d != 0 for d in range(2, int(n ** 0.5) + 1))

print(list(filter(eh_primo, range(1, 30))))

codigos = ["123", "ab1", "007", "", "4 5"]
print(list(filter(str.isdigit, codigos)))  # método pronto
```

SAÍDA

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
['123', '007']
```

- ✓ Regra com mais de uma etapa merece um `def` com nome que explique
- ✓ `filter(eh_primo, ...)` se lê quase como português
- ✓ Métodos prontos também servem: `str.isdigit` recusa vazio, letras e espaços

`all(...)` com uma expressão geradora para no primeiro divisor encontrado.

Tirando os *valores vazios*

filtro_none.py

```
respostas = ["sim", "", None, 0, "não", [], 42, False]

print(list(filter(None, respostas)))    # tira os "falsy"
```

SAÍDA

```
['sim', 'não', 42]
```

VALORES "FALSY" (CONTAM COMO FALSO)

`None` e `False`

`0` e `0.0`

`""`, `[]`, `{}`, `()` e outras coleções vazias

Com `None` no lugar da função, o `filter` testa o próprio item. Cuidado: o número `0` também sai, e às vezes ele é um dado válido.

O que *não* passou

filterfalse.py

```
from itertools import filterfalse

notas = [8.5, 4.0, 7.0, 5.5, 9.0]

def aprovado(nota):
    return nota >= 6

print("aprovados: ", list(filter(aprovado, notas)))
print("reprovados:", list(filterfalse(aprovado, notas)))
```

SAÍDA

```
aprovados:  [8.5, 7.0, 9.0]
reprovados: [4.0, 5.5]
```

- ✓ `itertools.filterfalse` faz o oposto: mantém os que dão **falso**
- ✓ Com a mesma função, separa os dois grupos sem escrever a regra invertida
- ✓ Evita erros como `nota < 6` num lugar e `nota >= 6` em outro

Filtros *encadeados*

dicionarios.py

```
produtos = [  
    {"nome": "Teclado", "preco": 250, "estoque": 4},  
    {"nome": "Mouse", "preco": 90, "estoque": 0},  
    {"nome": "Monitor", "preco": 899, "estoque": 2},  
    {"nome": "Cabo", "preco": 25, "estoque": 30},  
]  
  
disponiveis = filter(lambda p: p["estoque"] > 0, produtos)  
baratos = filter(lambda p: p["preco"] < 300, disponiveis)  
print([p["nome"] for p in baratos])
```

SAÍDA

```
['Teclado', 'Cabo']
```

- ✓ Cada item é um dicionário; a lambda olha o campo que interessa
- ✓ Um `filter` pode receber outro: primeiro "em estoque", depois "abaixo de 300"
- ✓ Como são iteradores, nenhuma lista intermediária é criada

O Monitor tem estoque, mas é caro; o Mouse é barato, mas está zerado.

Limpar, filtrar *e converter*

map_filter.py

```
entrada = "10, x, 25, , 7, 40"

partes = map(str.strip, entrada.split(","))
validos = filter(str.isdigit, partes)
numeros = map(int, validos)
print(sum(numeros))

# O mesmo com uma comprehension:
print(sum(int(p) for p in entrada.split(",")
          if p.strip().isdigit()))
```

SAÍDA

```
82
82
```

MAP (STRIP)
limpa espaços



FILTER (ISDIGIT)
fica só número



MAP (INT)
converte

- ✓ Mesma ideia do pipeline de geradores da aula 04
- ✓ A versão com comprehension faz tudo numa expressão
- ✓ As duas dão 82: escolha a que fica mais clara no seu código

Só os *e-mails válidos*

Enunciado: de uma lista de cadastros, mantenha só os e-mails com formato válido, usando `filter` e o padrão da aula 07.

Uma resposta possível

```
import re

cadastros = ["ana@site.com", "bia@", "caio@x.com.br",
            "", "dani site.com", "edu@empresa.org"]

PADRAO = re.compile(r"[\w.+~]+@[~\w-]+(\.[~\w-]+)+")

validos = list(filter(PADRAO.fullmatch, cadastros))
print(validos)
print(f"{len(validos)} de {len(cadastros)} são válidos")
```

SAÍDA

```
['ana@site.com', 'caio@x.com.br', 'edu@empresa.org']
3 de 6 são válidos
```

- ✓ `PADRAO.fullmatch` devolve um `Match` (verdadeiro) ou `None` (falso): serve direto no `filter`
- ✓ Desafio: mostre também os inválidos com `filterfalse`

Teste o que *ficou*

1. `list(filter(lambda x: x > 2, [1, 3, 5]))` vale:

A) [1]

B) [3, 5]

C) [False, True, True]

D) 2

B.

2. `list(filter(None, [0, 1, "", "a"]))` vale:

A) [0, ""]

B) [1, "a"]

C) [0, 1, "", "a"]

D) []

B.

3. A diferença entre `map` e `filter` é que `filter`:

A) Transforma cada item

B) Mantém ou descarta itens, sem alterá-los

C) Devolve uma lista

D) Só aceita números

B.

4. `filterfalse(f, itens)` devolve os itens em que:

A) `f` deu verdadeiro

B) `f` deu falso

C) `f` deu erro

D) Todos

B.

Fica o que *passa no teste*

`filter(função, iterável)` mantém os itens em que a função dá verdadeiro, sem alterá-los, e devolve um iterador preguiçoso. `filter(None, ...)` remove os valores falsy, `filterfalse` devolve o complemento, e filtros encadeados com `map` formam pipelines. Com regras simples, a comprehension com `if` faz o mesmo.

`filter(f, it)`

`None · falsy`

`filterfalse`

`map + filter`

`[x for x in it if ...]`

Próxima aula → 10 · Usando Decorator

Fontes: docs.python.org/3/library/functions.html#filter; docs.python.org/3/library/itertools.html#itertools.filterfalse; docs.python.org/3/library/stdtypes.html#truth-value-testing; docs.python.org/3/library/functions.html#all; docs.python.org/3/howto/functional.html, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.